

Databases, Auth, Deployment

Secrets & Environment Variables

Environment Variables

- `.env` file in project root — **never committed to git!**

```
DATABASE_URL=postgresql://user:pass@host:5432/db  
SUPABASE_URL=https://xyz.supabase.co  
SUPABASE_KEY=eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXLTJ0eS...  
PUBLIC_APP_NAME=My Cool App
```

- **Server-only** — only works in `+page.server.ts` , `+server.ts` , `hooks.server.ts` :

```
import { DATABASE_URL } from '$env/static/private';
```

- **Public** — works everywhere, including `.svelte` files:

```
import { PUBLIC_APP_NAME } from '$env/static/public';
```

Env Variable Rules

- **What goes in env vars:** database credentials, API tokens/keys, configurable endpoints, feature flags
- Variables **without** `PUBLIC_` prefix: server-only, SvelteKit errors if imported in client code
- Variables **with** `PUBLIC_` prefix: available everywhere, visible in the browser
- Add `.env` to `.gitignore` immediately
- On Vercel: set env vars in **Project Settings > Environment Variables**

Database Landscape

SQL Databases

- **Structured, relational, schemas enforced**

Database	Use case	Providers
SQLite	Prototypes, small apps	Turso, local file
PostgreSQL	Production apps	Neon, Supabase, Vercel Postgres
MySQL	Legacy	PlanetScale

- **When to use SQL:** structured data with relationships, joins, transactions

NoSQL Databases

- Flexible, schema-less or schema-optional

Database	Type	Providers
Redis	Key-value	Upstash, Redis Cloud
MongoDB	Document store	MongoDB Atlas

- **When to use NoSQL:** simple key-value lookups, caching, data that doesn't fit neatly into tables
- **Tip:** PostgreSQL with `jsonb` columns often eliminates the need for a separate NoSQL database

Backend as a Service (BaaS)

- Database + auth + API bundled together

Service	Database	Notes
Supabase	PostgreSQL	Auth, storage, realtime — best free tier
Firebase	Firestore	Google's BaaS, more lock-in
Convex	Custom	Reactive, real-time, great TypeScript DX
PocketBase	SQLite	Self-hosted, single binary

- When to use BaaS: you want to move fast and don't need full control

ORMs & File Storage

- **ORMs** — write TypeScript instead of raw SQL
 - **Drizzle**: lightweight, type-safe, SQL-like syntax — best for SvelteKit
 - **Prisma**: more opinionated, schema file, migrations, good DX
 - Use an ORM when working with SQL directly (not a BaaS)
- **Provider SDKs** — BaaS providers ship their own client libraries
 - `@supabase/supabase-js`, Firebase SDK, Convex client, etc.
 - No ORM needed — the SDK handles queries, auth, and more
- **File storage** — don't store large files in your database
 - Use S3-compatible storage: AWS S3, Cloudflare R2, Supabase Storage
 - Pattern: store file in S3, store the URL in your database

Connecting Supabase

Supabase Setup

1. Go to supabase.com — sign up with GitHub
2. **New Project** — pick a name, generate a DB password, choose closest region
3. Wait ~1 min for provisioning
4. **Settings > API** — copy the **Project URL** and **anon public** key
5. Paste into `.env` :

```
PUBLIC_SUPABASE_URL=https://xyz.supabase.co  
PUBLIC_SUPABASE_KEY=eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXZWQ0In...
```

Supabase Setup (cont.)

6. Install the SDK: `npm install @supabase/supabase-js`

7. Create a client helper:

```
// src/lib/supabase.ts
import { createClient } from '@supabase/supabase-js';
import { PUBLIC_SUPABASE_URL, PUBLIC_SUPABASE_KEY } from '$env/static/public';

export const supabase = createClient(PUBLIC_SUPABASE_URL, PUBLIC_SUPABASE_KEY);
```

8. **Table Editor** — create a table for your project's data, disable RLS for now

RLS (Row Level Security) — Postgres feature that restricts which rows a user can access. Supabase enables it by default because its API is exposed to the client. We disable it for now since we're calling Supabase from the server only.

Reading Data

```
// src/routes/posts/+page.server.ts
import { supabase } from '$lib/supabase';
import type { PageServerLoad } from './$types';

export const load: PageServerLoad = async () => {
  const { data: posts, error } = await supabase
    .from('posts')
    .select('*')
    .order('created_at', { ascending: false });

  if (error) {
    console.error('Failed to load posts:', error);
    return { posts: [] };
  }

  return { posts };
};
```

Writing Data

```
// In the same +page.server.ts
import type { Actions } from './$types';

export const actions: Actions = {
  default: async ({ request }) => {
    const formData = await request.formData();
    const title = formData.get('title') as string;
    const body = formData.get('body') as string;

    const { error } = await supabase
      .from('posts')
      .insert({ title, body });

    if (error) {
      return { success: false, message: error.message };
    }

    return { success: true };
  }
};
```

Exercise: Connect a Database

Exercise: Connect a Database

Goal: data is read from and written to a persistent database

1. Sign up for Supabase (free tier) and create a project
2. Get the URL and anon key, put them in `.env`
3. Create the `supabase` client in `src/lib/supabase.ts`
4. Create a table for your project's data in the Supabase dashboard
5. Replace in-memory data or public API fetch with a Supabase query
6. Add a form that inserts data into the table
7. **Verify:** data persists across page reloads and server restarts

Break

Auth Theory

Authentication vs Authorization

- **Authentication** (authn): who are you? → login
- **Authorization** (authz): what are you allowed to do? → permissions
- You always authenticate first, then authorize

Example: logging into GitHub = authentication. Having push access to a specific repo = authorization.

Password Auth vs OAuth

	Password Auth	OAuth
How	User creates username/password	User logs in via GitHub / Google / etc.
Pro	Simple, no third-party dependency	No passwords to store, provider handles security
Con	You handle security (hashing, resets, breaches)	Dependency on provider, more complex flow
Recommendation: use OAuth for most apps		

Token Format: Session ID vs JWT

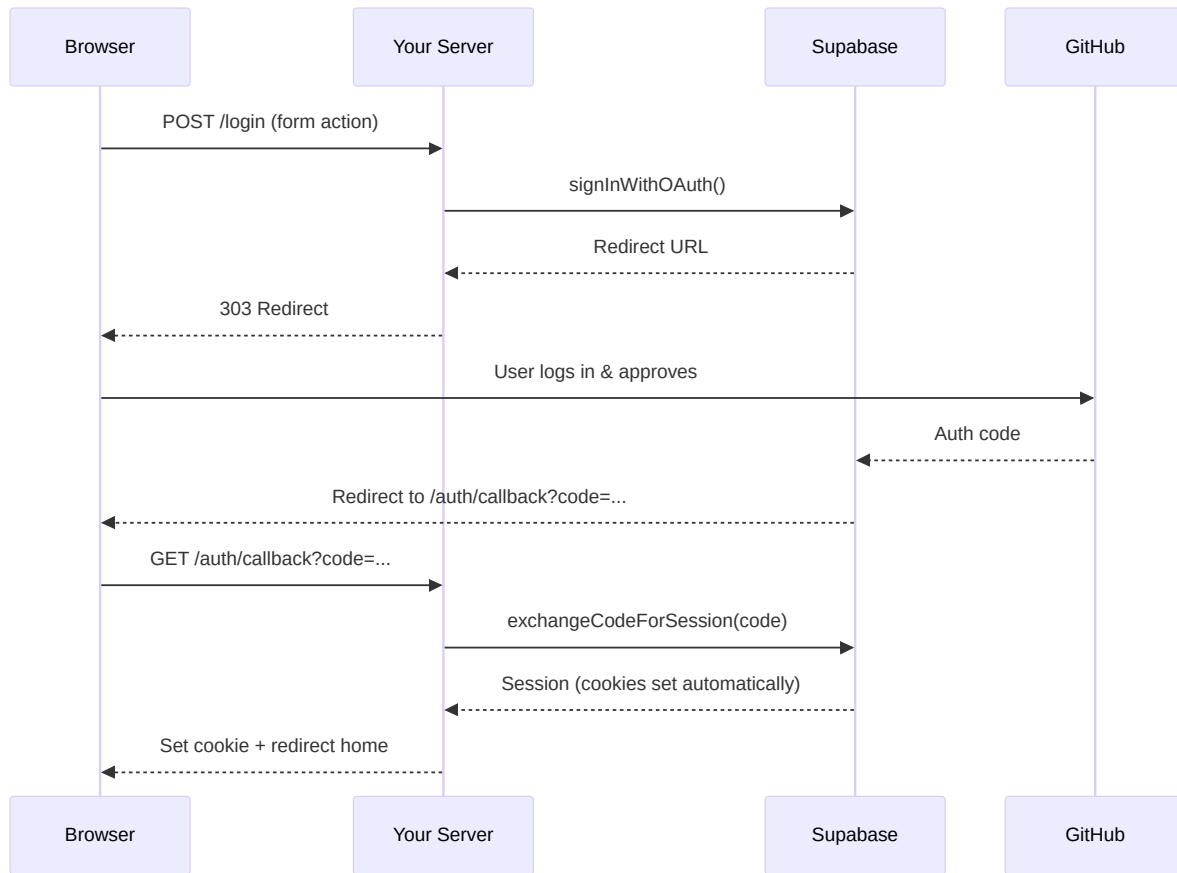
	Session ID	JWT (JSON Web Token)
What it is	Random string that references server-side data	Signed token that <i>contains</i> user data
Server state	Must store sessions (DB, Redis, memory)	Stateless — nothing stored on server
Revocation	Delete the session → instant revoke	Can't revoke until expiry
Size	Small (just an ID)	Grows with the data it carries

Token Transport: Cookie vs Header

	httpOnly Cookie	Authorization Header
Sent by	Browser, automatically on every request	JavaScript, manually via <code>fetch</code>
XSS safe?	Yes — JavaScript can't read it	No — token must be stored in JS (localStorage, variable)
Works with	Server-rendered pages, form actions, API calls	API calls only (SPAs, mobile apps)

Rule of thumb: use httpOnly cookies for server-rendered apps like SvelteKit

OAuth Login Flow



The Problem with Our Global Client

Our current `supabase` client is a **single global instance**:

```
// src/lib/supabase.ts - this worked fine for data queries
export const supabase = createClient(PUBLIC_SUPABASE_URL, PUBLIC_SUPABASE_KEY);
```

This breaks with auth because:

- **Auth state is per-user** — but a global instance is shared across all requests
- If user A logs in, the global client stores A's session — now user B sees A's data
- Sessions need to live in **cookies** (per-browser), not in a shared server variable
- The base `createClient` doesn't know how to read/write cookies

We need: a Supabase client that is created **per request** and uses **cookies** for session storage.

The Solution: `@supabase/ssr`

```
npm install @supabase/ssr
```

`@supabase/ssr` provides `createServerClient` — a Supabase client that:

- Is created **per request** (not a global singleton)
- Reads auth tokens from **cookies** (via `getAll`)
- Writes auth tokens to **cookies** (via `setAll`)
- Automatically refreshes sessions when tokens expire

But where do we create a per-request client? We need something that runs on **every request**, before any load function or action...

SvelteKit Middleware

hooks.server.ts

```
// src/hooks.server.ts - runs on EVERY request
import type { Handle } from '@sveltejs/kit';

export const handle: Handle = async ({ event, resolve }) => {
  console.log(`→ ${event.request.method} ${event.url.pathname}`);

  const response = await resolve(event);

  console.log(`← ${response.status}`);
  return response;
};
```

- `hooks.server.ts` runs **before** any load function, action, or API route
- `resolve(event)` hands the request to SvelteKit (routing, rendering, etc.)
- You can do work **before** (inspect request) and **after** (modify response)

event.locals

- `event.locals` is a **per-request object** — lives for one request, then disappears
- Set data in middleware → read it in load functions and actions

```
// src/hooks.server.ts
export const handle: Handle = async ({ event, resolve }) => {
  event.locals.user = { name: 'Jane Doe', email: 'jane@example.com' };
  return resolve(event);
};
```

```
// src/routes/+page.server.ts — locals is available in every route
export const load: PageServerLoad = async ({ locals }) => {
  return { user: locals.user };
};
```

Typing Locals

```
// src/app.d.ts - type your locals for autocomplete and safety
declare global {
  namespace App {
    interface Locals {
      user?: { name: string; email: string };
    }
  }
}
export {};
```

- Gives you autocomplete and type safety on `event.locals` everywhere
- SvelteKit picks up this file automatically

Supabase Auth

Setting Up the Server Client

```
// src/hooks.server.ts
import { createServerClient } from '@supabase/ssr';
import { PUBLIC_SUPABASE_URL, PUBLIC_SUPABASE_KEY } from '$env/static/public';
import type { Handle } from '@sveltejs/kit';

export const handle: Handle = async ({ event, resolve }) => {
  event.locals.supabase = createServerClient(
    PUBLIC_SUPABASE_URL, PUBLIC_SUPABASE_KEY, {
    cookies: {
      getAll: () => event.cookies.getAll(),
      setAll: (cookiesToSet) => {
        cookiesToSet.forEach(({ name, value, options }) =>
          event.cookies.set(name, value, { ...options, path: '/' })
        );
      }
    }
  });
  // continued on next slide...
  return resolve(event);
};
```

- The cookie wrapper bridges Supabase's session management with SvelteKit's cookie API
- `getAll` reads existing session cookies, `setAll` writes new ones (e.g. after login or token refresh)

safeGetSession Helper

```
// Still in hooks.server.ts – add after creating the client
event.locals.safeGetSession = async () => {
  const { data: { session } } = await event.locals.supabase.auth.getSession();
  if (!session) return { session: null, user: null };

  // getSession() reads from cookies without validating the JWT
  // getUser() makes a server call to validate it
  const { data: { user }, error } = await event.locals.supabase.auth.getUser();
  if (error) return { session: null, user: null };

  return { session, user };
};
```

Why not just `getSession()` ? It reads the JWT from cookies without validating it. A malicious user could tamper with the cookie. `getUser()` calls Supabase to verify the token is real.

Updating app.d.ts

```
// src/app.d.ts
import type { SupabaseClient, Session, User } from '@supabase/supabase-js';

declare global {
  namespace App {
    interface Locals {
      supabase: SupabaseClient;
      safeGetSession: () => Promise<{
        session: Session | null; user: User | null
      }>;
    }
  }
}

export {};
```

- Update the types to match what we now put on `locals`
- Every route gets autocomplete for `locals.supabase` and `locals.safeGetSession()`

Protecting Routes

```
// src/routes/+layout.server.ts
import { redirect } from '@sveltejs/kit';
import type { LayoutServerLoad } from './$types';

export const load: LayoutServerLoad = async ({ locals, url }) => {
  const { user } = await locals.safeGetSession();

  if (!user && !url.pathname.startsWith('/auth')
    && !url.pathname.startsWith('/login')) {
    redirect(302, '/login');
  }

  return { user: user ? { id: user.id, email: user.email ?? '' } : null };
};
```

- Use `locals.safeGetSession()` to get the validated user
- Protect a single route, or use a **layout** load to protect a group of routes

Auth Practical

GitHub OAuth App Setup

1. Go to GitHub: **Settings > Developer settings > OAuth Apps > New OAuth App**
2. Fill in the form:
 - **Application name:** anything (e.g. "My Svelte App")
 - **Homepage URL:** `http://localhost:5173`
 - **Authorization callback URL:** `https://YOUR_PROJECT.supabase.co/auth/v1/callback`
3. Copy the **Client ID** and **Client Secret**
4. In Supabase dashboard: **Authentication > Providers > GitHub**
5. Paste the Client ID and Secret, enable the provider

Login Page

```
<!-- src/routes/login/+page.svelte -->
<script lang="ts">
  import { enhance } from '$app/forms';
  let { form } = $props();
</script>

<h1>Login</h1>

{#if form?.error}
  <p class="text-red-600 mb-4">{form.error}</p>
{/if}

<form method="POST" action="?/github" use:enhance>
  <button class="rounded bg-gray-900 px-4 py-2 text-white">
    Sign in with GitHub
  </button>
</form>
```

- Just a form — no JavaScript imports, no browser client needed
- The form action on the server handles the entire OAuth flow

Login Server Action

```
// src/routes/login/+page.server.ts
import { fail, redirect } from '@sveltejs/kit';
import type { Actions } from '.$types';

export const actions: Actions = {
  github: async ({ locals }) => {
    const { data, error } = await locals.supabase.auth.signInWithOAuth({
      provider: 'github',
      options: { redirectTo: 'http://localhost:5173/auth/callback' }
    });

    if (error) return fail(400, { error: error.message });
    if (data.url) redirect(303, data.url);
  }
};
```

- `signInWithOAuth` on the server returns a redirect URL — it does **not** redirect automatically
- We use SvelteKit's `redirect()` to send the browser to GitHub
- `redirectTo` tells Supabase where to send the user after the GitHub handshake

Auth Callback

```
// src/routes/auth/callback/+server.ts
import { redirect } from '@sveltejs/kit';
import type { RequestHandler } from '.$types';

export const GET: RequestHandler = async ({ url, locals }) => {
  const code = url.searchParams.get('code');

  if (code) {
    await locals.supabase.auth.exchangeCodeForSession(code);
  }

  redirect(303, '/');
};
```

- Uses `locals.supabase` — the per-request server client from hooks
- `exchangeCodeForSession` exchanges the OAuth code **and** sets cookies automatically
- No manual `cookies.set()` needed — `@supabase/ssr` handles it via the `setAll` callback in hooks

Exercise: Add Login

Exercise: Add Login

Goal: a login flow that protects at least one route

Option A — Supabase Auth with `@supabase/ssr` (recommended):

- Install `@supabase/ssr`, set up `createServerClient` in `hooks.server.ts`
- Add `safegetSession` helper on `locals`
- Enable GitHub provider in Supabase dashboard
- Create login page with form action (server-side `signInWithOAuth`) and auth callback route
- Migrate data queries from `import { supabase }` to `locals.supabase`
- Protect routes via `locals.safegetSession()` in a layout load

Option B — Simple cookie auth (faster):

- Create a login form that checks a hardcoded password
- Set a session cookie on success via a form action
- Check the cookie in `hooks.server.ts`
- Protect a route

Deployment

Deploying with Vercel

The easiest option for SvelteKit:

```
npm install -g vercel  
vercel
```

Or connect via GitHub (recommended):

1. Push project to a GitHub repo
2. Go to vercel.com, import the repo
3. Vercel auto-detects SvelteKit — no config needed

Set environment variables in Vercel dashboard:

- Project Settings > Environment Variables
- Add `PUBLIC_SUPABASE_URL` , `PUBLIC_SUPABASE_KEY` , etc.
- These replace the `.env` file in production

Other Hosting Options

Platform	Notes
Netlify	Similar to Vercel, auto-detects SvelteKit
Cloudflare Pages	Edge deployment, very fast, free tier
Node server	<code>adapter-node</code> , deploy anywhere (Docker, VPS)
Static	<code>adapter-static</code> , fully pre-rendered, no server

Exercise: Deploy to Vercel

Exercise: Deploy to Vercel

Goal: a live URL you can share

1. Initialize git if needed: `git init && git add . && git commit -m "initial"`
2. Push to GitHub (create a repo, follow the instructions)
3. Import the repo on vercel.com
4. Set environment variables in the Vercel dashboard
5. Wait for the build, then open the live URL
6. **Test:** login works, data loads, navigation works
7. Share the URL!

Wrap-Up & Final Project

What We Covered

- **SvelteKit**

- File-based routing, layouts and pages
- Load functions and form actions
- Middleware and environment variables

- **Svelte**

- Client-side state: `$state`, `$derived`, `$effect`, `.svelte.ts` files
- Components, `$props`, snippets

- **Frontend**

- Semantic HTML, styling, fonts, icons

- **Backend**

- Authentication and middleware
- Databases
- Deployment

Final Project

- **Deadline:** April 16, 2026
- **Submission:** live URL (Vercel) + source code (GitHub repo) to daniel.allmer@axtesys.at
- **Grading:** functionality, code quality, effort & ambition, creativity

Resources:

- [Svelte docs](#)
- [SvelteKit docs](#)
- [Svelte tutorial](#)
- [Tailwind docs](#)
- [Supabase docs](#)

Questions?